

Comparative Study of Misbehavior Detection System for Classifying misbehaviors on VANET

Omessaad Slama¹, Bechir Alaya², Salah Zidi³ and Mounira Tarhouni⁴

Abstract—The purpose of the research on Vehicular Ad hoc Network (VANET) is to improve road safety, provide passenger comfort, and prevent accidents. Messages sent through VANETs are vulnerable to a variety of misbehaviors. Traditional techniques, like encryption, are ineffective since there is no purpose in being impervious to insider misbehavior. In this study, we propose an automatic learning method for detecting the misbehaving message deliver through a vehicle in VANETs using machine learning and deep learning techniques. To evaluate the performance of these techniques, we used the first publicly available Vehicular Misbehavior Dataset VeReMi. In this paper, we will compare the performance of ML and DL in VANET for detecting and classifying misbehavior messages.

Index Terms—Misbehavior Detection, VeReMi dataset, Machine Learning (ML), Deep Learning (DL), VANETs, Binary Classification.

I. INTRODUCTION

Researchers are paying close attention to vehicular ad-hoc networks (VANETs) as an inspirational technology for providing comfort to drivers, improving road efficiency, and reducing the number of accidents [1]. VANET is a wireless

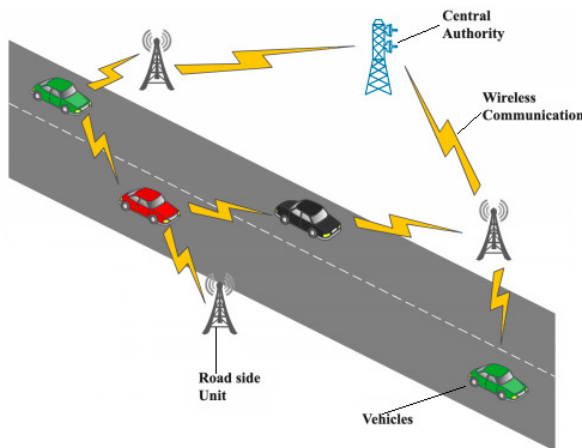


Fig. 1. Vehicular Ad-hoc Networks (VANET)

network that connects many vehicles by utilizing OBUs (On-Board Units) to communicate with other vehicles and RSUs (Road-Side Units) [2] Fig.1. However, the number of

¹O.Slama, ³S.Zidi and ⁴M.Tarhouni are with Hatem Bettaher Laboratory (IRESCOMATH), University of Gabes, Tunisia slama.oumsaad@gmail.com; salah.zidi@yahoo.fr ; mounira.tarhouni@isimg.tn

²B.Alaya is with the Department of Management Information Systems and Production Management, College of Business and Economics, Qassim University, Saudi Arabia b.alaya@qu.edu.sa

communications between Vehicle to Infrastructure (V2I) and Vehicle to Vehicle (V2V) is growing, resulting in an increase

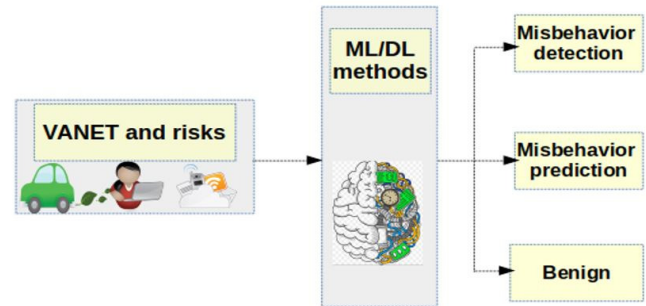


Fig. 2. Model ML/DL based MDS for VANET

in cyber-security attacks in VANETs [3] like replay attacks, false and alternate messages, and fake alerts. As a result, it is critical to look for security solutions to avoid these types of attacks. Each vehicle exchanges the Basic Safety Messages (BSMs) specified in the SAE J2735 standard with other nodes in VANETs. Every 100 ms each vehicle diffuses a BSM containing their current position, speed, acceleration, heading, and a temporarily pseudonym to identify the sender. Nevertheless, a malicious node diffuses a false position report in order to create a ghost node and fake alerts... The Public Key Infrastructure delivers digital certificates that serve as a signature for the exchanged messages. These signatures verify the message as defined by the IEEE 1609.2 protocol [4], but they do not guarantee its correctness, therefore there is a greater need to implement an effective misbehavior detection system (MDS) to identify and correctly classify misbehavior in VANETs. MDS solutions have been created using Artificial Intelligence (AI) approaches for this purpose, According to Venture capitalist Frank Chen's overview, he says: "Artificial intelligence is a set of algorithms and intelligence to try to mimic human intelligence. Machine learning is one of them, and deep learning is one of those machine learning techniques". DL and ML are subsets of AI that help machines in making decisions about the type of traffic. The components of a typical MDS based on ML/DL approaches are represented in Fig. 2. There are several types of machine and Deep Learning classifiers, each with its own set of properties that set it apart from the others. We are interested in detecting and classifying misbehavior in this study, as well as determining the best efficient machine learning/deep learning classifier for MDS using the VeReMI dataset. Five machine learning algorithms were included:

K-Nearest Neighbors (KNN), Random Forest (RF), Naive Bayes (NB), Logistic Regression (LR), Decision Tree (DT) and three Deep Learning algorithms: Artificial Neural Network (ANN), Convolutional Neural Network (CNN), and Long Short Term Memory (LSTM). Our purpose is not to integrate the detection performances of the ways, but rather to provide a detection system measure to the community by investigating the common Machine Learning and Deep Learning methodologies. Our contributions on this topic are summarized here. Section II presents some related works. Section II introduces our suggested Machine Learning and Deep Learning Classifiers. Section III describes the dataset used in this research, as well as the suggested approach. Section IV presents and discusses the results of the proposed approach. Section V concludes the work.

II. RELATED WORKS

The simulations have been utilized in previous studies to study misbehavior in Vehicular Ad hoc Networks, and a detailed description of some innovative studies in the development of misbehavior detection is provided below. Rens et al. [5] published the first public Vehicular Reference Misbehavior Dataset (VeReMi) in 2018 to provide a reference dataset for any researcher to add new attacks and compare their results to others'. They successfully completed simulations of the Luxembourg SUMO Traffic (LuST) scenario [6]. They evaluate various detectors and conclude that precision-recall is the most useful metric for comparison. The misbehavior detection system, on the other hand, can identify known misbehavior but not unknown one. Josef et al. enhanced the VeReMi dataset [7] in 2020 by including a realistic physical error model as well as a set of attacks on the VeReMi dataset. They provided benchmark detection metrics by utilizing a large number of local detectors and a basic misbehavior detection algorithm. Khan et al. [8] released an article about detecting Malicious Nodes in VANETs. They introduced an innovative method called as Detection of Malicious Nodes (DMN) in VANETs. They carried out the experiment using a network simulator. Grover et al. published a study [9] on machine learning for detecting various misbehaviors in VANETs. Among the algorithms used were Random Forest, IBK, Naive Bayes, AdaBoost1, and J-48. Random Forest and J-48, on the other hand, generated the best results. In [10], Sharanya et al. proposed an idea called Classifying Malicious Nodes in VANETs using SVM with Modified Fading Memory(MFM). The algorithms employed in this work were SVM and MFM, both of which are semi-supervised learning algorithms with Receiver Operator Characteristic Curve (ROC) values of 98%. Steven et al. presented a study [11] on detection and classification of location spoofing attacks on the VeReMi dataset using KNN and SVM machine learning classifiers, their results were acceptable. They provided a framework for a system that detects and classifies misbehavior. Singh et al. proposed another study [12] on a ML algorithm to detect position falsification attacks in VANETs. They used Logistic Regression and SVM machine learning techniques to examine several

feature extractions that yielded varying accuracy outcomes. According to their findings, SVM outperforms Logistic Regression. Waleed et al. proposed Optimized Node Clustering in VANETs using Meta-Heuristic algorithms in [13]. The grasshoppers' optimization-based node clustering approach was proposed in this paper. In circumstances when node density was unpredictable, the proposed method decreased network overhead. Zahra et al. [14] have out a research on misbehaving node identification in VANETs. In comparison to SVM-based, Dempster-Shafer-based, and averaging-based detection methods. The Svm algorithm has the best accuracy.

III. MATERIALS AND METHODS

In this study, we utilized the VeReMi Extension dataset. The dataset contains a set of malfunctions related to the faulty position, speed, acceleration, heading values caused by a faulty OBU or vehicle sensors and a whole of attacks that outcome of vehicles by forwarding incorrect data. In each type of misbehavior, we find two subsets: one subset in high traffic time (7h-9h) and another in poor traffic time (14h-16h) and a mix of all misbehaviors in all day (0h-24h). The different types of attacks and faults in the VeReMi extension dataset are shown in table 1:

A. Pre-processing

In this section, we follow the proposed flowchart shown in Fig. 3 to classify ML/DL tasks on large data.

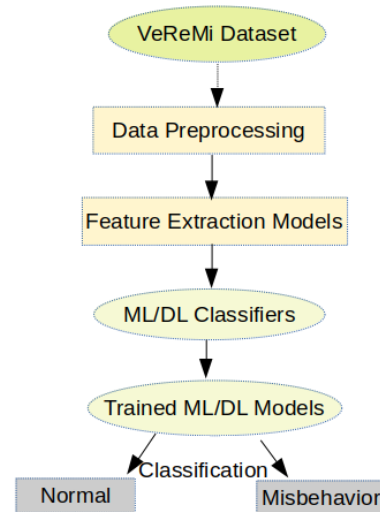


Fig. 3. Model ML/DL based Misbehavior Detection System for VANET

The VeReMi dataset is a genuine dataset for detecting misbehavior in vehicular communication. The principal features of the data field frames are utilized to learn the model of the MDS. The data fields contain information about the vehicle such as reception time; transmission time; sender ID; message-ID; attacker type; pseudo-identity; (x, y, z coordinates of the vehicle for: position, velocity, acceleration, heading, position noise, velocity noise, acceleration noise and heading noise) and we added a labeled class type as 'normal'

TABLE I
ATTACKS AND FAULTS WITHIN VeReMi DATASET

Misbehavior	Type	Description
Constant Position	Fault	The vehicle diffuses a faulty constant position (X,Y), the fixed values are disposed at every introduction of novel malfunctioning node.
Constant Position Offset	Fault	The vehicle diffuses its true position with a constant offset ($\Delta X, \Delta Y$)
Constant Speed	Fault	The vehicle diffuses a faulty constant speed.
constant Speed Offset	Fault	The vehicle diffuses its real speed with a constant offset.
Random Position	Fault	The vehicle diffuses a faulty limited random position.
Random Position Offset	Fault	The vehicle diffuses its true position with a limited random offset.
Random Speed	Fault	The vehicle diffuses a faulty limited random speed.
Random Speed Offset	Fault	The vehicle diffuses its true speed with a limited random offset.
Delayed messages	Attack	The vehicle diffuses its correct messages however are forward with retard from existence (Δt).
Eventual Stop	Attack	The malicious node diffuses a null speed and a fixed positions thereby reproducing a sudden stop.
DoS	Attack	The vehicle forwards messages with a greater frequency than the limit described in IEEE or ETSI standards. This cause an overhead on the transmission channel, so the channel can be inaccessible by other vehicles.
Dos Random	Attack	The vehicle executes a DoS attack whereas together randomizing all the V2X messages data.
DoS Random Sybil	Attack	The attacker executes a DoS Random with changing her identity in any forward message to deflect detection.
Disruptive	Attack	The malicious vehicle replays the precedent received messages diffused by their random neighbor. This can prevent genuine messages from being broadcasted and flood the network.
DoS disruptive	Attack	The attacker executes DoS and Disruptive attacks at the same time.
DoS Disruptive Sybil	Attack	The malicious vehicle executes a Dos Disruptive attack and at the same time modifying their identity in every forwarding data.
Data Replay	Attack	The attacker sends the data precedently received in distinction to an unique destination neighbor. This message was signed using the attacker's certificate.
Data Replay Sybil	Attack	The attacker executes a Data Replay attack modifying their pseudonym on each new selected destination to avoid detection.

or 'misbehavior' to identify the type of vehicle. After the data has been loaded into the programming environment, we start by pre-processing our data through cleaning data, delete every repetition and missed value in order to build an efficient misbehavior detection system. The data is split at random into 70% for training and 30% for testing.

B. Feature Extraction Methods

Since the VeReMi database have a high number of features, there are some irrelevant attributes that may have a detrimental influence on the model's effectiveness in detecting abnormal traffic. As a result, we use three distinct feature extraction methods named 'Model 1', 'Model 2', and 'Model 3' to eliminate the undesirable features from our dataset. In 'Model 1', we tested the impact of each attribute on the labeled class attribute by using a Python script to identify which database attributes are more relevant than others, and then remove irrelevant features. In 'Model 2', we used f-test in one-way Analysis of Variance (ANOVA) [15]. In 'Model 3', Recursive Feature Elimination (RFE) [16] a Wrapper method, was utilized. After removing irrelevant data from the supplied dataset using the three Models, we will evaluate the performance of the VeReMi dataset using ML and DL classification algorithms.

C. Implementation information

Every algorithm is trained in Python on a NVIDIA GPU and a 2.50 GHz Intel Core i5-10300H CPU with 8 GB of RAM. We study and test eight algorithms of Machine Learning (RF, KNN, NB, LR, DT) and Deep Learning (ANN, CNN, LSTM). The hyperparameters of each algorithms examined, as well as their values, will be presented below. When all of the evaluation metrics are taken into account, the KNN [17] classifier produces a great performance for

the value $k = 3$. The Gaussian naïve bayes classifier is used for the naïve bayes [18] classifier. The Gini index is used in the decision tree [19] model. On Random Forest [20], the number of estimators is set at 300. The training procedure for CNN [21] is a Stochastic Gradient Descent (SGD) optimizer with momentum equal to 0.9 and an initial learning rate of 0.01. The number of training epochs is set to 100 and the batch size is set to 25. For training, the Adaptive Delta (Adadelta) optimizer is utilized. The hidden layer count is set to four. The activation function is the Rectified Linear Unit (RELU). A dropout with a factor of 0.1 is employed during CNN training to avoid data overfitting. The adaptive moment estimation "Adam" optimizer with a learning rate of 0.001 is utilized for LSTM [22] training, and the "binary crossentropy" function is employed for compilation. The batch size is set to 25, and the number of epochs is set to 100. The activation function utilized is sigmoid. The hidden layer count is set to three. The batch size is set to 25, and the number of epochs is set to 100. The activation function utilized is sigmoid. The hidden layer count is set to three. For ANN [23] training, a learning rate of 0.002 and an Adam optimizer are utilized. The batch size is set to 25, and the number of epochs is set to 50. The hidden layer count is set to three. The activation function RELU is utilized.

IV. RESULTS AND DISCUSSION

In this part, we use the confusion matrix to assess the performance of machine learning and deep learning methods for MDS using Precision (P) and Recall (R) defined below. The confusion matrix illustrates the relationship between the actual and predicted values. Table 2 illustrates the confusion matrix to calculate the evaluated metrics:

- **Precision:** shows the system's competency to differen-

TABLE II
CONFUSION MATRIX

		Predicted result	
		Negative	Positive
Actual Result	Negative	True Negative (TN)	False Positive (FP)
	Positive	False Negative (FN)	True Positive (TP)

tiate between normal and misbehaving vehicles, i.e. a low precision signifies the classifiers yields a lot of FP.

$$Precision = \frac{TP}{TP + FP} \quad (1)$$

- **Recall:** shows the classifier's competency to detect a misbehaving vehicle, i.e. a low recall signifies an attack is hard to detect.

$$Recall = \frac{TP}{TP + FN} \quad (2)$$

The algorithm correctly predicts misbehavior in TP, whereas the classifier incorrectly classed the misbehaving as normal in FN. In FP, the model incorrectly identified the data instances as a malicious, however in TN, the classifiers correctly predict the negative samples. Models in cybersecurity are evaluated using precision and recall metrics. When precision and recall conditions are better and higher, the model tests are better. However, these metrics are in several conditions paradoxical only in certain job requirements. The purpose of this section is to apply and compare three feature selection approaches ('Model 1', 'Model 2', and 'Model 3') for some ML/DL models in each type of misbehavior (attack and fault) for the VeReMi dataset, first, both the high density subset (07h - 09h) and the low density subset (14h -16h). We can see that the associated misbehavior data was used just once for each of these tests. For example, attack type Dos data was not used in attack Disruptive inside the train and test ensembles. In Tables 3 and 4, we select the models with the highest accurate classification rate among the three models that will be employed in a detection framework and we present the results in high density because the traffic density and the time of day have no effect on detection performance: the two subgroups ((07h – 09h) and (14h – 16h)) to each kind of misbehavior have the same detection rate.

A. Detection Per Type of Misbehavior in ML

Table 3 shows the results of Machine Learning classifiers including Random Forest, KNN, Naive Bayes, Logistic Regression and Decision Tree, as well as their best feature extraction models in high density.

'Model 3' (RFE) outperforms 'Models 1' and 'Model 2' in Random Forest classifiers as shown in Table 3. Misbehaviors with larger Precision and Recall are 'ConstSpeed', 'DosRandom' and 'DosRandomSybil', with 1 in both, so the system can distinguish genuine vehicles from misbehaving vehicles. While the 'DataReplay' attack is difficult to detect with an accuracy of 0.52 and a recall of 0.47, it means a large amount of FP. 'Model 3' of the KNN classifier produces the best

results; for example, in 'DosRandom' attack, Precision and Recall are 0.9999 and 0.9941, respectively. Because precision and recall in 'DataReplay' and 'Disruptive' attacks were low, detection became challenging. In a 'DataReplay' attack, the attacker replays data from another vehicle; these attacks are designed to trick the detection system into misidentifying honest vehicles as attackers. We notice that the Naive Bayes classifiers yield a lot in FP and the detection is difficult in many misbehaviors such as: 'RandomPosOffset', 'Dos', 'DataReplay', 'ConsPosOffset', 'DataReplaySybil', 'Disruptive'...

In certain misbehavior scenarios, the Logistic Regression classifiers generate poor Precision and Recall outputs, making it difficult to detect misbehaving nodes and discriminate between malicious and normal vehicles. We notice that 'Model 1' gives the best results in Naive Bayes and Logistic Regression algorithms. 'Model 3' performs better in the Decision Tree classifier, with a better Precision and Recall in the 'ConstSpeed' fault and poor results in the 'DataReplay' and 'Disruptive' attacks.

According to the evaluation metrics, we conclude that the type of misbehavior has an effect on detection quality. When we use feature extraction approaches, we discover that the accuracy varied between models and classifiers, when the model choose the relevant features the performance of the algorithm increase.

In machine learning classifiers, the identification of a misbehaving vehicle is difficult with disruptive attacks and attacks that replay the data of another vehicle, It is critical to discover a solution to these types of attacks in future research with new progressive approaches in misbehavior detection systems.

Based on many tests mentioned above, it is clear that Random Forest, KNN, and Decision Tree classifiers produce good results in 'Model 3' witch is a wrapper method that produce the greatest results in terms of providing a detection system and it is more resistant to overfitting.

In terms of precision and false-positive weights, Random Forest classifiers outperform KNN and DT classifiers. We should also mention that the Random Forest classifiers are the quickest throughout the train and test phases.

B. Detection Per Type of Misbehavior in Deep Learning

Table 4 illustrates the results of the ANN, CNN, and LSTM classifiers in Deep Learning.

'Model 1' is the best model in the ANN algorithm, and the higher Precision and Recall are 1 and 0.9992 respectively in the 'DosRandom' attack, but a bad result in 'Disruptive' attack by 0.5259 in Precision and 0.0278 in Recall.

In the CNN algorithm, 'Model 1' outperforms 'Models 2 and 3'; the best result is in 'DosRandom' attack with 0.9942 and 0.9796 in Precision and Recall, respectively, and a poor detection in 'Disruptive' attack and 'RandomPosOffset' fault.

'Model 3' performs well in LSTM classifiers, with a high Precision and Recall values of 0.9981 and 0.9924 in 'DosRandomSybil', respectively.

TABLE III
CLASSIFICATION RESULTS IN ML ALGORITHMS

	RF: Model 3		KNN: Model 3		NB: Model 1		LR: Model 1		DT: Model 3	
Id Misbehavior	P	R	P	R	P	R	P	R	P	R
ConstPos	1.00	0.98	0.97	0.90	0.69	0.25	0.58	0.04	0.98	0.98
ConstPosOffset	0.99	0.97	0.89	0.81	0.40	0.05	0.40	0.05	0.97	0.97
ConstSpeed	1.00	1.00	0.99	0.97	0.99	0.85	1.00	0.92	0.997	0.998
ConstSpdOffset	1.00	0.99	0.99	0.97	0.55	0.53	0.91	0.85	0.992	0.990
RandomPos	0.99	0.94	0.96	0.77	0.67	0.26	0.53	0.04	0.96	0.95
RandomPosOffset	0.99	0.79	0.76	0.65	<i>0.87</i>	<i>0.00</i>	<i>0.00</i>	<i>0.00</i>	0.81	0.80
RandomSpeed	1.00	0.99	0.99	0.96	0.99	0.85	1.00	0.93	0.994	0.993
RandomSpdOffset	1.00	0.99	0.99	0.89	0.94	0.22	<i>0.00</i>	<i>0.00</i>	0.98	0.98
DataReplay	<i>0.52</i>	<i>0.47</i>	0.51	0.49	0.41	0.11	<i>0.00</i>	<i>0.00</i>	<i>0.43</i>	<i>0.46</i>
DataReplaySybil	0.73	0.72	0.68	0.73	0.39	0.08	<i>0.00</i>	<i>0.00</i>	0.63	0.72
DosRandom	1.00	1.00	0.999	0.994	0.97	0.96	0.99	0.97	0.999	0.999
DosRandomSybil	1.00	1.00	0.99	0.99	0.96	0.95	0.99	0.97	0.99	0.98
Dos	0.96	0.98	0.84	0.93	0.56	0.89	0.56	0.99	0.90	0.97
DosDistructive	0.69	0.76	0.66	0.78	0.57	0.80	0.56	0.98	0.64	0.71
DosDistructiveSybil	0.90	0.91	0.84	0.90	0.52	0.61	0.50	0.14	0.83	0.90
Distructive	0.54	0.51	<i>0.49</i>	<i>0.52</i>	0.42	0.07	<i>0.00</i>	<i>0.00</i>	0.43	0.49
DelayedMessage	0.99	0.78	0.87	0.80	0.84	0.55	1.00	0.52	0.83	0.84
GridSybil	0.99	0.91	0.92	0.91	0.81	0.85	0.64	0.99	0.92	0.93
EventualStop	0.99	0.86	0.94	0.88	0.70	0.81	0.20	0.00	0.89	0.90
Average	0.91	0.87	0.86	0.83	0.7	0.51	<i>0.52</i>	<i>0.44</i>	0.85	0.87

The highest metrics are bolded, and the lowest metrics are italicized.

TABLE IV
CLASSIFICATION RESULTS IN DL ALGORITHMS

	ANN: Model 1		CNN: Model 1		LSTM: Model 3	
Id Misbehavior	P	R	P	R	P	R
ConstPos	0.9491	0.8234	0.9372	0.4002	0.9477	0.5501
ConstPosOffset	0.8903	0.7467	0.4817	0.0045	0.7374	0.4090
ConstSpeed	0.9990	0.9930	0.9997	0.9179	0.9985	0.9264
ConstSpdOffset	0.9949	0.9936	0.9641	0.8658	0.9446	0.8823
RandomPos	0.9457	0.8181	0.9141	0.4343	0.8698	0.6165
RandomPosOffset	0.5435	0.0381	<i>0.0000</i>	<i>0.0000</i>	<i>0.0000</i>	<i>0.0000</i>
RandomSpeed	0.9987	0.9948	0.9997	0.9261	0.9974	0.9380
RandomSpdOffset	0.9969	0.9855	0.9824	0.3155	0.8595	0.3037
DataReplay	0.6598	0.0653	0.8015	0.0028	0.6505	0.0465
DataReplaySybil	0.6311	0.0681	0.7685	0.0063	0.5960	0.0135
DosRandom	1.0000	0.9992	0.9942	0.9796	0.9907	0.9841
DosRandomSybil	0.9999	0.9990	0.9938	0.9737	0.9981	0.9924
Dos	0.6107	0.7825	0.5583	0.9848	0.5680	0.8919
DosDistructive	0.6053	0.7888	0.5721	0.8954	0.5863	0.8442
DosDistructiveSybil	0.6397	0.6004	0.5966	0.3936	0.5680	0.5797
Distructive	<i>0.5259</i>	<i>0.0278</i>	<i>0.0000</i>	<i>0.0000</i>	<i>0.0000</i>	<i>0.0000</i>
DelayedMessage	0.9886	0.5326	1.0000	0.5247	1.0000	0.5229
GridSybil	0.9335	0.7958	0.8611	0.8288	0.8984	0.8085
EventualStop	0.9818	0.7693	0.8804	0.7694	0.8328	0.6771
Average	0.8384	0.6759	0.7263	0.5145	0.7674	0.5606

The highest metrics are bolded, and the lowest metrics are italicized.

When the Deep Learning classifiers are compared, we discover that ANN generates the best results and that 'Model 1' produce the best model in features extraction method.

V. CONCLUSION AND FUTURE WORK

In this study, we tackled the problem of identifying and classifying misbehaving vehicles in VANETs. We proposed and studied the application of some Machine Learning and Deep Learning approaches using the VeReMi dataset. According to the experimental results, ML approaches are

more effective and promising than DL methods in classifying misbehavior. In the ML classifiers: Random Forest, Decision Tree, and KNN function well since they have a low false alarm rate for detection per type of misbehavior and we notice that the Wrapper methods for Feature extraction is the best to pick the relevant attributes. As future work, we elaborate to combine both Misbehavior Detection System and Misbehavior Prevention System in one system including signature-based with approaches to detect misbehaviors via the Misbehavior Prevention System in VANET.

REFERENCES

- [1] J. Santa, F. Pereñíguez, A. Moragón and A. F. Skarmeta, "Experimental evaluation of CAM and DENM messaging services in vehicular communications," *Transportation Research Part C: Emerging Technologies*, vol. 46, p. 98-120, 2014.
- [2] M. Nidhal, J. B. Othmen and H. Mohamed, "Survey on VANET security challenges and possible cryptographic solutions," *Vehicular Communications*, vol. 1, no 2, p. 53-66, 2014.
- [3] T. Alladi, V. Chamola, B. Sikdar, and K.-K. R. Choo, "Consumer iot: Security vulnerability case studies and solutions," *IEEE Consumer Electronics Magazine*, vol. 9, no 2, p. 17-25 2020.
- [4] K. J. Jung and H. S. Phil, "Study on the Privacy-Preserving Vehicular PKI in Autonomous Driving Environments," 2016.
- [5] V. W. Rens, T. Lukaseder, and K. Frank, "Veremi: A dataset for comparable evaluation of misbehavior detection in vanets," *arXiv preprint arXiv:1804.06701*, 2018.
- [6] L. Codecá, R. FRANK, S. FAYE and T. Engel, "Luxembourg sumo traffic (lust) scenario: Traffic demand evaluation," *IEEE Intelligent Transportation Systems Magazine* vol. 9, no. 2, pp. 52-63, 2017.
- [7] K. Joseph, W. Michael, V. W. Rens, A. Kaiser, P. Urien, and K. Frank, "VeReMi extension: A dataset for comparable evaluation of misbehavior detection in VANETs." *CC 2020-2020 IEEE International Conference on Communications (ICC)*. IEEE, 2020.
- [8] U. Khan, S. Agrawal, and S. Silakari, "Detection of Malicious Nodes (DMN) in Vehicular Ad-Hoc Networks," *International Conference on Information and Communication Technologies (ICICT)*, pp. 965-972, 2015.
- [9] J. Grover, N. K. Prajapati, L. Vijay, G. M. Singh, "Machine Learning Approach for Multiple Misbehavior Detection in VANET," *International conference on advances in computing and communications*. Springer, Berlin, Heidelberg, 22-24 July, 2011.
- [10] S. Sharanya, and S. Karthikeyan, "Classifying Malicious Nodes In Vanets Using Support Vector Machines With Modified Fading Memory," *ARNP Journal of Engineering and Applied Sciences*, 2017
- [11] SO. Steven, P. SHARMA, and J. PETIT, "Integrating plausibility checks and machine learning for misbehavior detection in VANET," *17th IEEE International Conference on Machine Learning and Applications (ICMLA)*, IEEE, 2018.
- [12] P. K. SINGH, S. GUPTA, R. VASHISTHA, and N. Sukumar, "Machine learning based approach to detect position falsification attack in vanets," *International Conference on Security Privacy*. Springer, Singapore, pp. 166-178, 2019.
- [13] A. Waleed, F. K. Muhammad, A. Farhan, M. Muazzam, A. Staish, N. Yunyoung et al. "Optimized node clustering in VANETs by using meta-heuristic algorithms," *Electronics*, vol. 9, no 3, pp. 394, 2020.
- [14] Z. S. Mohammadi, K. Mizanian, M. A. Sarram and S. Goli, "Misbehavior Node Detection in Vehicular ad-hoc Networks: A survey, With Special Emphasis on Multihop Broadcast Protocols," *Researcher*, vol. 9, no. 1, pp. 41-46, 2017.
- [15] J. D. F. HABBEMA, and J. HERMANS, "Selection of variables in discriminant analysis by F-statistic and error rate," *Technometrics*, vol. 19, no. 4, pp. 487-493, 1977.
- [16] X. CHEN and J. C. JEONG, "Enhanced recursive feature elimination," *Sixth International Conference on Machine Learning and Applications*, pp. 429-435, 2007.
- [17] K. Ismo et al, "Outlier detection using k-nearest neighbour graph", 2004.
- [18] M. Swarnkar, N. Hubballi, "OCPAD: One class Naive Bayes classifier for payload based anomaly detection", *Expert Syst. Appl.*, vol. 64, pp. 330-339, 2016.
- [19] S. Himani, K. Sunil, "A survey on decision tree algorithms of classification in data mining", *Int J Sci Res IJSR*, vol. 5, pp. 2094-2097, 2016.
- [20] A.L. Buczak and E. A. Guven, "survey of data mining and machine learning methods for cyber security intrusion detection", *IEEE Commun. Surv. Tutor*, vol. 18, pp. 1153-1176, 2015.
- [21] A. Javed, N. MOUSTAFA, H. KHURSHID, et al. "A review of intrusion detection systems using machine and deep learning in internet of things: Challenges, solutions and future directions", *Electronics*, vol. 9, no 7, pp. 1177, 2020..
- [22] R. BARZEGAR, M.T. AALAMI and J. ADAMOWSKI, "Short-term water quality variable prediction using a hybrid CNN-LSTM deep learning model", *Stochastic Environmental Research and Risk Assessment*, pp. 1-19, 2020.
- [23] J. Goldarag, Y. MohammadZadeh and A. Ardakani, "A Fire risk assessment using neural network and logistic regression", *J Indian Soc Remote Sens*, vol. 44, pp. 885-894, 2016.